تغییر یابد تا آدرس مبنای جدید را منعکس نماید. زمانی که فشرده‌سازی امکان‌پذیر است، بایستی هزینه‌ی آن تعیین شود. ساده‌ترین الگوریتم فشرده‌سازی آن است که تمامی پردازه‌ها به یک طرف حافظه و تمامی حفره‌ها به طرف دیگر حافظه جابجا شوند که با این کار، حفره‌ی بزرگی از حافظه فراهم می‌شود. این طرح می‌تواند گران تمام شود.

راه‌حل ممکن دیگر مسأله‌ی پارگی خارجی آن است که اجازه بدهیم فضای آدرس منطقی پردازه‌ها غیرهمجوار گردد، لذا پردازه بتواند فضای فیزیکی حافظه را در هر جا که شد در اختیار بگیرد. این راهبرد مورد استفاده در *صفحه‌بندی*، رایج‌ترین تکنیک مدیریت حافظه در سیستم‌های کامپیوتری است. ما صفحه‌بندی را در بخش بعدی بیان می‌کنیم.

پارگی یک مسأله‌ی عمومی در محاسبات است که می‌تواند در هر جایی که می‌بایست بلاک‌هایی از داده‌ها را مدیریت کنیم، اتفاق بیفتند. این موضوع را با عمق بیشتر در فصول مدیریت ذخیره‌سازی (فصل ۱۱ تا فصل ۱۵) بحث و بررسی می‌کنیم.

۹-۳ صفحه‌بندی

مدیریت حافظه‌ای که تاکنون بحث شده لازم دارد که فضای آدرس فیزیکی یک پردازه همجوار باشد. اکنون **صفحه‌بندی** را معرفی می‌کنیم، یک طرح مدیریت حافظه‌ای که اجازه می‌دهد فضای آدرس فیزیکی یک پردازه غیرهمجوار گردد. صفحه‌بندی از پارگی خارجی و نیاز به فشرده‌سازی مرتبط اجتناب می‌کند، دو مسئله‌ای که تخصیص همجوار حافظه با آن درگیر است. صفحه‌بندی به‌خاطر مزایای متعددی که دارد در اَشکال گوناگونش در اکثر سیستم‌عامل‌ها، از کارگزارهای بزرگ گرفته تا دستگاه‌های موبایل مورد استفاده قرار می‌گیرد. صفحه‌بندی با همکاری میان سیستم‌عامل و سخت‌افزار کامپیوتر پیاده‌سازی می‌شود.

۹-۳-۱ روش پایه‌ای

روش پایه‌ای پیاده‌سازی صفحه‌بندی، شکستن حافظه‌ی فیزیکی به بلاک‌هایی با اندازه‌ی ثابت به نام **قاب‌ها**[1] و شکستن حافظه‌ی منطقی به بلاک‌هایی با اندازه‌ی یکسان به نام **صفحات** می‌باشد. زمانی که پردازه‌ای اجرا می‌شود، صفحات آن از ذخیره‌ی پشتیبان در داخل هر یک از قاب‌های موجود حافظه بارگذاری می‌گردند. ذخیره‌ی پشتیبان به بلاک‌هایی با اندازه‌ی ثابت تقسیم می‌شود که با قاب‌های حافظه یا خوشه‌هایی از قاب‌های متعدد، هم اندازه هستند. این ایده‌ی به ظاهر ساده کارکردی فوق‌العاده و دسته‌های وسیعی دارد. برای مثال، هم‌اکنون فضای آدرس منطقی به صورت کامل از فضای آدرس فیزیکی جدا شده است، طوری که یک پردازه می‌تواند یک فضای آدرس منطقی ۶۴ بیتی، حتی در سیستمی داشته باشد که کمتر از 2^{64} بایت حافظه‌ی فیزیکی دارد.

هر یک از آدرس‌های تولید شده توسط پردازنده به دو قسمت تقسیم می‌شوند: **شماره‌ی صفحه (p)** و **افست صفحه (d)**.

1. Frames

شماره‌ی صفحه	افست صفحه
p	d

شماره‌ی صفحه به‌عنوان یک ایندکس به **جدول صفحه‌ی** هر پردازه استفاده می‌شود. این موضوع در شکل ۹–۸ نمایش داده می‌شود. جدول صفحه شامل آدرس مبنای هر قاب در حافظه‌ی فیزیکی است و افست، مکانی از قاب است که رجوع می‌شود. بنابراین، آدرس مبنای قاب با افست صفحه ترکیب می‌شود تا آدرس حافظه‌ی فیزیکی را معین کند. مدل صفحه‌بندی حافظه در شکل ۹–۹ نشان داده می‌شود.

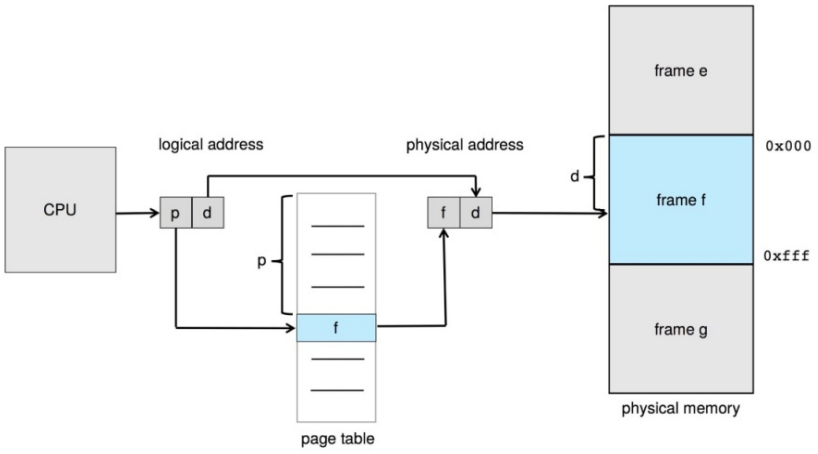
MMU برای ترجمه‌ی یک آدرس منطقی تولید شده توسط پردازنده به یک آدرس فیزیکی، گام‌های زیر را دنبال می‌کند:

۱- شماره‌ی صفحه‌ی p را استخراج کرده و به‌عنوان یک ایندکس به جدول صفحه استفاده کن.

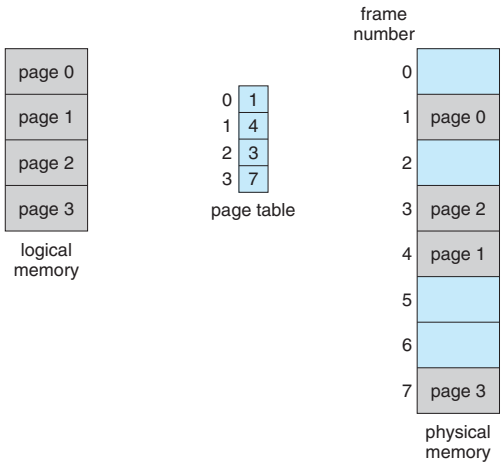
۲- شماره‌ی قاب f را از جدول صفحه استخراج کن.

۳- شماره‌ی صفحه‌ی p در آدرس منطقی را با شماره‌ی قاب f جایگزین کن.

چون افست d تغییر نمی‌کند، جایگزین نشده، و اکنون، شماره‌ی قاب و افست است که آدرس فیزیکی را تشکیل می‌دهند.



شکل ۹–۸ سخت‌افزار صفحه‌بندی.

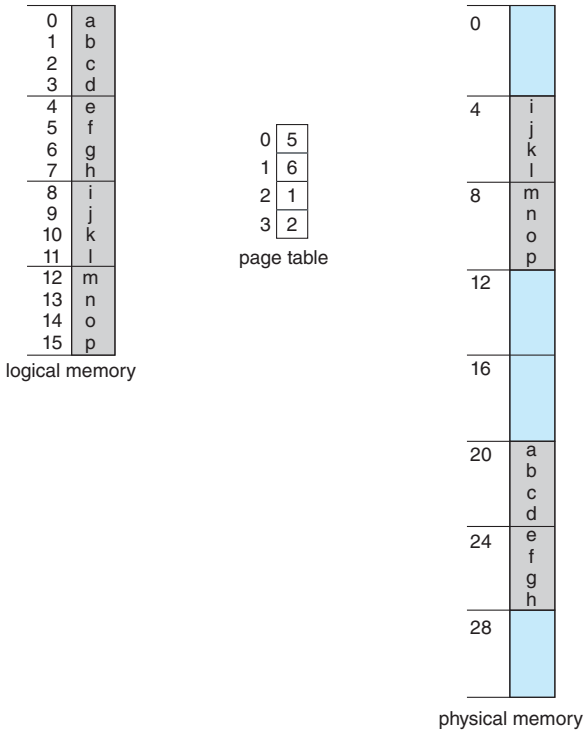


شکل ۹-۹ مدل صفحه‌بندی حافظه‌ی منطقی و فیزیکی.

اندازه‌ی صفحه (مثل اندازه‌ی قاب) توسط سخت‌افزار تعریف می‌شود. اندازه‌ی هر صفحه توانی از ۲ است که بسته به معماری کامپیوتر، نوعاً برای هر صفحه بین ۴ کیلوبایت و ۱ گیگابایت در تغییر است. انتخاب توانی از ۲ برای اندازه‌ی صفحه باعث می‌شود که ترجمه‌ی آدرس منطقی به شماره‌ی صفحه و افست صفحه فوق‌العاده آسان‌تر شود. اگر اندازه‌ی فضای آدرس منطقی 2^m و اندازه‌ی هر صفحه 2^n بایت باشد، در این صورت m - n بیت با ارزش آدرس منطقی، معرف شماره‌ی صفحه و n بیت کم ارزش آن، افست صفحه را مشخص خواهند کرد. بنابراین، آدرس منطقی به صورت زیر است:

شماره‌ی صفحه	افست صفحه
p	d
$m - n$	n

به عنوان مثالی عینی‌تر (هرچند در مقیاسی کوچک)، حافظه‌ای را که شکل ۹-۱۰ نشان می‌دهد در نظر بگیرید. در اینجا، در آدرس منطقی، n برابر ۲ و m برابر ۴ می‌باشد. با استفاده از اندازه‌ی صفحات ۴ بایتی و یک حافظه‌ی فیزیکی ۳۲ بایتی (۸ صفحه)، نشان می‌دهیم که چگونه دید برنامه‌نویس از حافظه می‌تواند به حافظه‌ی فیزیکی نگاشته شود. آدرس منطقی ۰، صفحه‌ی ۰ با افست ۰ می‌باشد. با ایندکس به جدول صفحه در می‌یابیم که صفحه‌ی ۰ در قاب ۵ قرار گرفته است. بنابراین، آدرس منطقی ۰ به آدرس فیزیکی ۲۰ (= ۰ + (۴ × ۵)) نگاشته می‌شود. آدرس منطقی ۳ (صفحه‌ی ۰ با افست ۳) به آدرس فیزیکی ۲۳ (= ۳ + (۴ × ۵)) نگاشته می‌شود. آدرس منطقی ۴، صفحه‌ی ۱ با افست ۰ می‌باشد که صفحه‌ی ۱، مطابق جدول صفحه به قاب ۶ نگاشته شده است. بنابراین، آدرس منطقی ۴ به آدرس فیزیکی ۲۴ (= ۰ + (۴ × ۶)) نگاشته می‌شود. آدرس منطقی ۱۳ به آدرس فیزیکی ۹ نگاشته می‌شود.



شکل ۹-۱۰ مثال صفحه‌بندی برای یک حافظه‌ی ۳۲ بایتی با صفحات ۴ بایتی.

ممکن است متوجه شده باشید که صفحه‌بندی خود شکلی از تغییر مکان پویاست. هر یک از آدرس‌های منطقی توسط سخت‌افزار صفحه‌بندی به یک آدرس فیزیکی پیوند می‌خورند. استفاده از صفحه‌بندی مشابه بکارگیری جدولی از ثبات‌های مبنا (یا تغییر مکان) برای هر یک از قاب‌های حافظه می‌باشد.

زمانی که از طرح صفحه‌بندی استفاده می‌کنیم، هیچ‌گونه پارگی خارجی نخواهیم داشت: هر قابی که آزاد باشد می‌تواند به هر پردازه‌ای که نیاز دارد تخصیص یابد. با این وجود، امکان دارد مقداری پارگی داخلی داشته باشیم. توجه دارید که قاب‌ها به صورت واحد تخصیص می‌یابند. اگر نیازهای حافظه‌ی یک پردازه مضربی از اندازه‌ی صفحات نباشد، *آخرین* قاب تخصیصی پردازه به طور کامل پر نمی‌شود. برای مثال، با اندازه‌ی صفحات ۲۰۴۸ بایتی، پردازه‌ای با اندازه‌ی ۷۲۷۶۶ بایت، به ۳۵ صفحه و ۱۰۸۶ بایت نیاز خواهد داشت. تخصیص ۳۶ قاب برای این پردازه، پارگی داخلی ۹۶۲ = ۱۰۸۶ – ۲۰۴۸ بایتی را به دنبال دارد. در بدترین حالت، نیاز پردازه می‌تواند n صفحه و فقط یک بایت باشد. در اینجا، هر n + ۱ قاب تخصیص می‌یابند که پارگی داخلی حاصل از آن، تقریباً یک قاب کامل خواهد بود.

اگر اندازه‌ی پردازه مستقل از اندازه‌ی صفحه باشد، به ازای هر پردازه انتظار داریم به طور متوسط، نیم‌صفحه پارگی داخلی به وجود بیاید. با این ملاحظه، صفحات با اندازه‌های کوچک‌تر مطلوب‌ترند. با این وجود، سربار تعداد واردهای جدول صفحه بالا می‌رود و این سربار با

افزایش اندازه‌ی صفحات کاهش می‌یابد. زمانی که میزان داده‌های انتقالی بزرگ‌تر می‌شوند I/O دیسک نیز کارآمدتر خواهد بود (فصل ۱۱). در حالت کلی، اندازه‌ی صفحات با بزرگ‌ترشدن پردازه‌ها، مجموعه‌ی داده‌ها و حافظه‌ی اصلی رشد داشته‌اند. امروزه، صفحات نوعاً در اندازه‌های ۴ کیلوبایت و یا ۸ کیلوبایت می‌باشند و حتی برخی سیستم‌ها از صفحات با اندازه‌ی بزرگ‌تر نیز پشتیبانی می‌کنند. برخی از پردازنده‌ها و سیستم‌عامل‌ها حتی از اندازه‌ی صفحات متعدد نیز پشتیبانی می‌کنند. برای نمونه، ویندوز ۱۰ در سیستم‌های x86-64 از اندازه‌ی صفحات ۴ کیلوبایت و ۲ مگابایت پشتیبانی می‌کند. لینوکس نیز از دو اندازه‌ی صفحه پشتیبانی می‌کند: اندازه‌ی صفحه‌ی پیش‌فرض (نوعاً ۴ کیلوبایت) و یک اندازه‌ی صفحه‌ی بزرگ‌تر وابسته به معماری که **صفحات بسیاربزرگ** نامیده می‌شوند.

بدست‌آوردن اندازه‌ی صفحه در سیستم‌های لینوکس

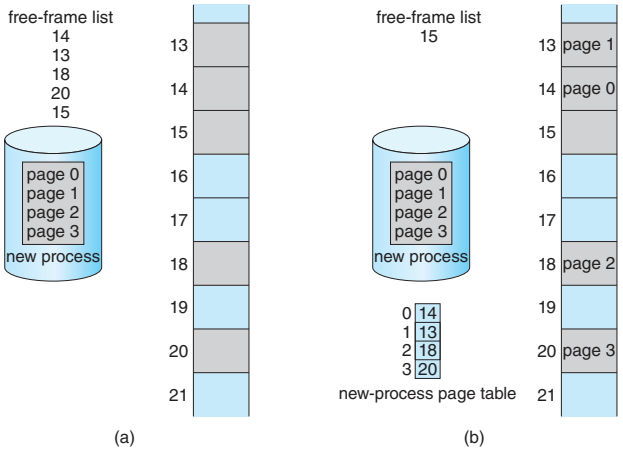
در یک سیستم لینوکس، اندازه‌ی صفحه مطابق معماری تغییر کرده و شیوه‌های متعددی برای بدست‌آوردن اندازه‌ی صفحه وجود دارند. یک رویکرد، استفاده از فراخوانی سیستمی getpagesize() است. راهبرد دیگر واردکردن فرمان زیر در خط فرمان است:

```
getconf PAGESIZE
```

هر یک از این دو تکنیک، اندازه‌ی صفحه را به صورت یک عدد با واحد بایت بر می‌گردانند.

در بیشتر موارد، در یک پردازنده‌ی ۳۲ بیتی، هر یک از وارده‌های جدول صفحه، ۴ بایت طول دارد، اما این اندازه هم می‌تواند تغییر کند. یک وارده‌ی ۳۲ بیتی می‌تواند به یکی از 2^{32} قاب صفحه‌ی فیزیکی اشاره کند. اگر اندازه‌ی قاب‌ها ۴ کیلوبایت (2^{12} بایت) باشد، در این صورت سیستمی با وارده‌های ۴ بایتی می‌تواند حافظه‌ی فیزیکی 2^{44} بایتی (یا ۱۶ ترابایتی) را آدرس‌دهی کند. در اینجا، بایستی توجه داشته باشیم که اندازه‌ی حافظه‌ی فیزیکی در یک سیستم حافظه‌ی صفحه‌بندی شده نوعاً با اندازه‌ی حداکثر منطقی یک پردازه فرق دارد. همچنان که صفحه‌بندی را با عمق بیشتری بررسی می‌کنیم، اطلاعات دیگری را معرفی خواهیم کرد که بایستی در وارده‌های جدول صفحه نگهداری شوند. این اطلاعات، تعداد بیت‌های در دسترس آدرس‌دهی قاب‌صفحات را کاهش می‌دهد. بنابراین، یک سیستم با وارده‌ی جدول صفحه‌ی ۳۲ بیتی می‌تواند حافظه‌ی فیزیکی کمتری از حداکثر ممکن آدرس‌دهی کند.

لحظه‌ای که پردازه‌ای برای اجرا به سیستم وارد می‌شود، اندازه‌اش برحسب صفحه برآورد می‌شود. هر یک از صفحات پردازه به یک قاب نیاز دارند. بنابراین، اگر پردازه n صفحه لازم داشته باشد، بایستی حداقل n قاب از حافظه در دسترس باشند. اگر n قاب موجود بود به همین پردازه‌ای که آمده، تخصیص می‌یابند. اولین صفحه‌ی پردازه در یکی از قاب‌های تخصیص‌یافته بارگذاری شده و شماره‌ی قاب آن در جدول صفحه برای این پردازه قرار داده می‌شود. صفحه‌ی بعدی در قاب دیگری بارگذاری شده و شماره‌ی قاب آن در جدول صفحه گذاشته می‌شود و الی آخر (شکل ۹-۱۱).



شکل ۹-۱۱ قاب‌های آزاد. (a) پیش از تخصیص و (b) پس از تخصیص.

یک وجه مهم صفحه‌بندی، تفکیک روشن دید حافظه‌ی برنامه‌نویس از حافظه‌ی فیزیکی واقعی می‌باشد. برنامه‌نویس، حافظه را به صورت فضای منفردی می‌بیند که تنها برنامه‌ی او را شامل است. برنامه‌ی کاربر به واقع در کل حافظه‌ی فیزیکی پراکنده شده است که برنامه‌های دیگری نیز وجود دارند. اختلاف دید برنامه‌نویس از حافظه و حافظه‌ی فیزیکی واقعی توسط سخت‌افزار ترجمه‌ی آدرس از بین می‌رود. آدرس‌های منطقی به آدرس‌های فیزیکی ترجمه می‌شوند. این نگاشت از دید برنامه‌نویس پنهان بوده و توسط سیستم‌عامل کنترل می‌شود. توجه داشته باشید که طبق تعریف، پردازه‌ی کاربر نمی‌تواند حافظه‌ای را که مالکیت آن را ندارد دستیابی کند. آن هیچ راهی برای آدرس‌دهی حافظه‌ی خارج از جدول صفحه‌اش را ندارد و جدول صفحه نیز تنها شامل صفحاتی است که پردازه مالک آنهاست.

از آنجایی که سیستم‌عامل حافظه‌ی فیزیکی را مدیریت می‌کند، بایستی به جزئیات تخصیص حافظه‌ی فیزیکی—یعنی، کدام قاب‌ها تخصیص یافته‌اند، کدام قاب‌ها آزادند، تعداد کل قاب‌ها و الی آخر—آگاهی داشته باشد. این اطلاعات عموماً در یک ساختمان‌داده‌ی منفرد سطح سیستمی به نام **جدول قاب** نگه‌داری می‌شوند. جدول قاب برای هر یک از قاب‌صفحات فیزیکی یک وارده دارد که نشان می‌دهد آیا قاب آزاد است و یا اگر تخصیص‌یافته، به کدام پردازه (یا پردازه‌هایی) تخصیص پیدا کرده است.

به علاوه، سیستم‌عامل بایستی مطلع باشد که پردازه‌های کاربران در فضای کاربر عملیات دارند و تمامی آدرس‌های منطقی باید نگاشته شوند تا آدرس‌های فیزیکی را تولید نمایند. اگر کاربری یک فراخوانی سیستمی صادر کرده (برای مثال، برای انجام I/O) و آدرسی را هم به عنوان پارامتر (برای نمونه، یک بافر) ارسال می‌کند، آن آدرس بایستی نگاشته شود تا آدرس فیزیکی صحیح تولید شود. سیستم‌عامل برای هر یک از پردازه‌ها همچنان که کپی شمارنده‌ی دستورالعمل و محتویات ثبات‌ها را نگه می‌دارد یک کپی از جدول صفحه نیز نگه می‌دارد. هر بار که سیستم‌عامل، نگاشت آدرس‌های منطقی به آدرس‌های فیزیکی را لازم دارد، از این کپی جدول